# Efficient Bound Propagation Applied to Fuzzy Trees for Verification of Breast Cancer Classification

Jared Burton[1[0009-0004-5827-3199]] and Kelly Cohen[2[0000-0002-8655-1465]]

[1,2] Department of Aerospace Engineering, University of Cincinnati, Cincinnati, OH, USA
[1] burtonjd@mail.uc.edu, [2] cohenky@ucmail.uc.edu

**Abstract.** Lack of robustness is a common issue exhibited by many machine learning models. Consequently, a means of assessing model robustness is a useful design tool. This work uses rigorous verification techniques to assess the robustness of fuzzy trees trained on the Breast Cancer Wisconsin (Original) dataset. Breast cancer classification as malignant or benign clearly benefits from a robust classifier. A constrained implementation of Adaptive Neuro-Fuzzy Inference Systems (ANFIS) is used to facilitate both verification and gradient-based training of each tree. The resulting fuzzy trees are checked for classification robustness at each data point when the input is subjected to bounded perturbations. Robustness is assessed by each of two verification approaches, CROWN and α-CROWN. These provide bounds on model outputs with varying levels of conservativeness; α-CROWN iteratively improves the bounds produced by CROWN. It is shown that higher nominal performance does not equate to greater robustness. Tradeoffs are examined between model robustness, model size, and compute time required for verification.

**Keywords:** Fuzzy Inference System, Fuzzy Tree, Verification.

## 1 Introduction

It is widely understood that many machine learning (ML) and artificial intelligence (AI) models lack explainability, interpretability, and robustness. The lack of explainability and interpretability can sometimes be mitigated by choosing an appropriate model, such as a Fuzzy Inference System (FIS). Decisions made by such a model are more readily explained and interpreted. On the other hand, it is not obvious how to design a more robust model or verify that it is, indeed, more robust. In this context, verification involves algorithmically generating mathematical proof that some desired condition is met. For many ML tasks, high nominal performance may be sufficient. For many others, however, robust & verified performance is necessary. In this work, breast cancer classification is used as a benchmark problem which benefits from both high nominal accuracy and high off-nominal performance, i.e., robustness.

Many verification approaches have been developed for a variety of ML models; very few have been applied to FISs. In other work, Satisfiability Modulo Theories (SMT) solvers have been used to verify FIS adherence to requirements which are specified in first-order logic formulas [1]. The effectiveness and efficiency of this approach has been demonstrated on a variety of benchmarks including output con-

straint violation and Lyapunov stability checking [1]. However, this approach to verification is not readily made generative. In other words, model performance can be checked against a requirement, but model performance cannot be described directly. This limitation, in addition to poor worst-case computational complexity of SMT-based verification, motivates alternative approaches such as bound propagation. This is discussed in greater detail below.

### 1.1 Fuzzy Trees

A single FIS with a full rule base scales poorly with the number of inputs. For example, a FIS with 5 membership functions per input and $N$ inputs will have $5^N$ rules. This exponential growth in rules with many inputs leads to a reduction in explainability and greater computational complexity. FIS design, either manually or via numerical optimization, also becomes more difficult.

Cascading fuzzy trees (Fig. 1) are a promising alternative which have been successfully employed by many researchers to solve a variety of problems [2,3,4,5]. Instead of a single FIS which processes all inputs and produces the final output, cascading fuzzy trees use a feedforward network consisting of multiple FISs to perform a computation. Each FIS may operate on a subset of the input features. Additionally, each FIS may take the outputs from other FISs as inputs. After perhaps many intermediate FISs, model outputs may be generated by a single or multiple FISs in combination.

Cascading multiple FISs together mitigates the exponential growth of the number of rules. Ideally, a fuzzy tree is also designed to enhance explainability. It may be possible to assign human-interpretable labels to intermediate FIS outputs; these can be used for debugging and design assistance. If the design of the tree is well-suited to the problem at hand, the structure chosen for the tree will also produce a good inductive bias when learning from data or simulation.

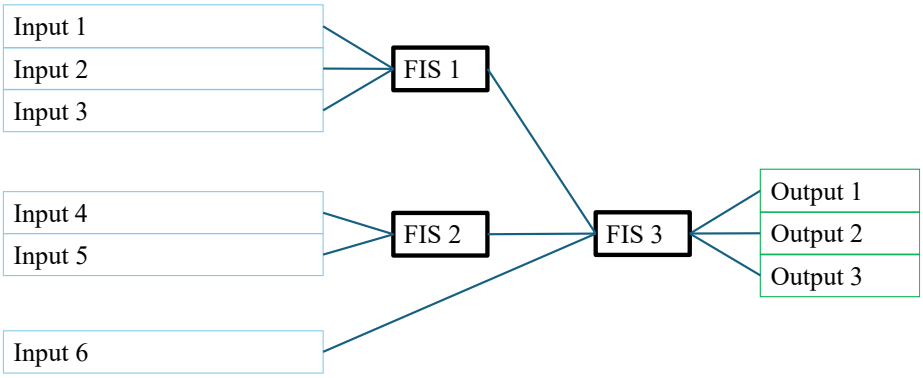


**Fig. 1.** Example of Cascading Fuzzy Tree with 6 Inputs, 3 Outputs, and 3 FISs

### 1.2 ANFIS

An Adaptive Neuro-Fuzzy Inference System (ANFIS) [6, 7] provides a feedforward neural network representation of a Takagi-Sugeno-Kang (TSK) [8] type FIS. Each of the operations which constitute a FIS is encapsulated by a separate layer (Fig. 2). Like fuzzy trees, ANFIS architectures have been successfully employed to solve a variety of problems in many fields. The FIS representation provided by ANFIS need not correspond to a full rule base. In this way, ANFIS has also been used to overcome the exponential scaling induced by a full rule base.

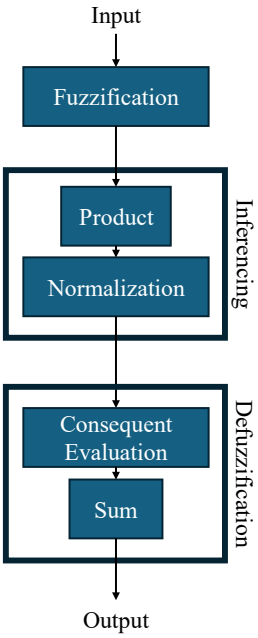


**Fig. 2.** Generic ANFIS Architecture and Layers

In this work, an ANFIS architecture is used in combination with fuzzy trees. First, a fuzzy tree is designed such that it is composed of FISs with full rule bases. Next, an equivalent ANFIS architecture is generated for each FIS. Then, the individual ANFIS architectures are substituted into the original tree structure. This yields a feedforward network which is equivalent to a fuzzy tree.

The ANFIS representation of a fuzzy tree offers several benefits. Automatically, this representation inherits all the benefits of a fuzzy tree. Additionally, this representation allows the engineer to take advantage of many modern machine learning libraries which perform automatic gradient calculations and produce a computational graph encapsulating the model. As per usual, the automatic gradient calculations are useful for learning from data and simulation traces. The computational graph representation is useful for verification.

### 1.3 Bound Propagation

Bound propagation is an approach to verification that seeks to approximate the set of possible outputs which may be produced by a ML model, given a set of possible inputs. Typically, in the case of a computational model with real inputs, the input set is given by a bounded perturbation about an input vector. For the bound propagation to be sound, an approximation of the output set ($\Omega_{approx}$) must be a superset of the true output set ($\Omega$). In other words, $\Omega \subseteq \Omega_{approx}$. Such approximations lend themselves to *incomplete verification*, in which there is no guarantee that every output in $\Omega_{approx}$ has a corresponding input within the input set. Incomplete verification tends to yield conservative results. *Complete verification*, on the other hand, computes the exact output set $\Omega$ either directly or iteratively. In an iterative approach to complete verification, $\Omega_{approx}$ converges to $\Omega$.

Recent developments in neural network verification have produced novel bound propagation approaches for both incomplete and complete verification. Techniques such as CROWN [9] and α-CROWN [10] have enabled efficient incomplete verification of large neural networks and general computational graphs. CROWN computes output bounds directly and α-CROWN iteratively improves on those bounds. Similarly, the α,β-CROWN [11] algorithm extends the previous approaches to make complete verification of large neural networks computationally tractable. Consequently, α,β-CROWN has won the Verification of Neural Networks Competition (VNN-COMP) [12] for the past 4 years (2021-2024). In general, bound propagation techniques have been shown to be faster than alternative approaches such as the SMT-based verification [9,10,11]. This is especially true for large numbers of requirements, e.g. checking robustness of a large model under many different conditions.

In this work, CROWN and α-CROWN are used for incomplete verification of fuzzy trees. Open-source implementations of both methods are provided in the Python library "auto_LiRPA: Automatic Linear Relaxation based Perturbation Analysis for Neural Networks" [13]. auto_LiRPA provides incomplete verification for general computational graphs defined using the PyTorch library [14]. Consequently, the ANFIS representations of fuzzy trees in this work are created using PyTorch. The unique contribution of this work is the application of bound propagation techniques to fuzzy trees and initial results for a relevant benchmark problem.

## 2 Methodology

### 2.1 Breast Cancer Dataset

The Breast Cancer Wisconsin (Original) dataset [15] was used in this work for proof of concept. The dataset is sourced from the University of California Machine Learning Repository and was originally provided by the University of Wisconsin Hospitals, Madison and Dr. William H. Wolberg. The purpose of this dataset is classification of potential breast cancer as malignant or benign; the dataset has served as a classification benchmark for a variety of ML approaches. The dataset includes 9 features and 699 instances. For sake of simplicity, instances with missing values are discarded for

the remainder of this work. The 9 features used as input to the classifier are titled clump thickness, uniformity of cell size, uniformity of cell shape, marginal adhesion, single epithelial cell size, bare nuclei, bland chromatin, normal nucleoli, and mitoses. After removing instances with missing values, the dataset contains 444 benign cases and 239 malignant cases.

### 2.2 Fuzzy Tree Selection

Two different fuzzy trees are proposed for solving the breast cancer classification problem. The trees are trained on the data and analyzed separately, for comparison. The first tree, model 1 (Fig. 3), is reproduced from [4] where a 94.96% test classification accuracy was achieved. Note that only the tree structure was inherited from prior work; the individual FISs were defined and trained independently. Model 1 consists of 7 FISs total. FISs 1-4 operate on the input features, and each FIS produces a single intermediate feature. FISs 5 and 6 operate on the collection of intermediate features produced by FISs 1-4. Finally, FIS 7 operates on the intermediate features produced by FISs 5 and 6 to produce 2 outputs corresponding to the 2 classes. The final classification belongs to the maximum output.

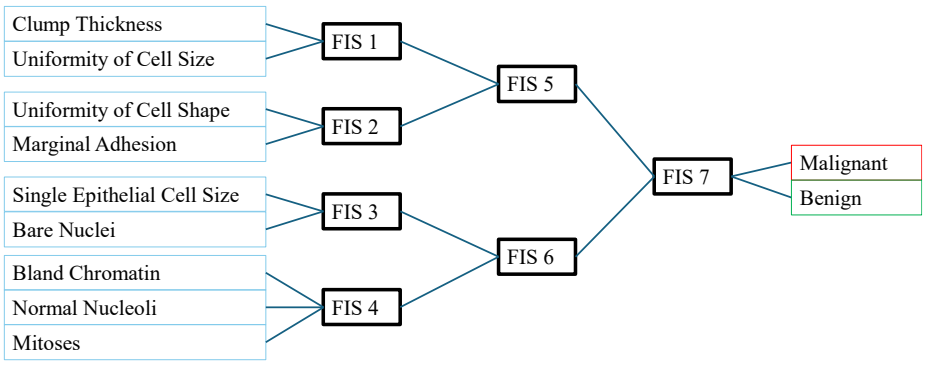


**Fig. 3.** Model 1's Fuzzy Tree

The second tree, model 2 (Fig. 4), consists of only 4 FISs organized into 2 layers rather than 3, as in model 1. The tradeoff achieved by having fewer layers is more inputs per FIS. All FISs in model 2 have 3 inputs whereas all but one of the FISs in model 1 have 2 inputs. Each aspect of the tree structure including number of FISs, number of inputs per FIS, and total number of layers are expected to affect the verification results. Each FIS in both models has 5 membership functions per input.

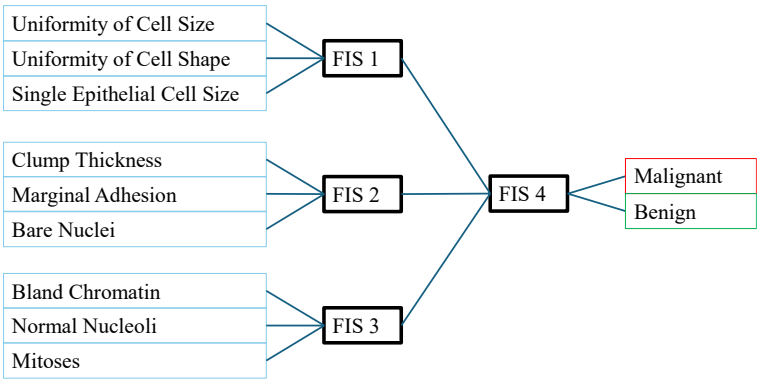


**Fig. 4.** Model 2's Fuzzy Tree

### 2.3 ANFIS Implementation

**FIS Constraints.** In this work, several constraints are imposed on each of the FISs that constitute the fuzzy trees. The membership functions defined over a given input consist only of left-shoulder, right-shoulder, and triangular types. The membership functions are always arranged such that the left-most one is a left-shoulder, the right-most one is a right-shoulder, and all intermediates are triangular. Ruspini partitioning [16] is used to restrict corners and centers of adjacent membership functions. For example, the center of a triangular membership function will coincide with both the right corner of the left-adjacent and left corner of the right-adjacent membership functions. Additionally, each FIS is a 0-order TSK with a full rule base. A multi-output FIS is implemented with multiple rule bases (1 per output) defined over the same input membership functions.

These constraints serve 2 purposes. First, 0-order consequents were chosen to ease training, reduce the capacity for overfitting, and are expected to result in tighter verification bounds. Second, the membership function constraints are sufficient to simplify the ANFIS implementation in a way that facilitates verification. Specifically, these constraints remove the need for a normalization layer. It has been shown in [1] that the normalization denominator is always equal to 1 when FISs are subject to these constraints. A normalization would layer loosen the verification bounds because it involves dividing by a number which, in principle, may be very close to 0. The computed bounds on the normalization denominator may even contain 0. As a result, the ANFIS implementation used in this work does not include a redundant normalization layer.

**Membership Function Representation.** In this work, each membership function type can be easily computed as a sum of weighted ReLU activations. This simplifies the incorporation of membership functions into the verification algorithms used here. Consider the triangular membership function $\mathrm{tri}(x; l, c, r)$ with input $x$, left corner $l$, right corner $r$, and center $c$. With the ReLU function definition provided in Equation

1, tri can be defined as in Equation 2. The left-shoulder $\mathrm{lshlder}(x; c, r)$ with center $c$ and right corner $r$ and the right-shoulder $\mathrm{rshlder}(x; l, c)$ with left corner $l$ and center $c$ are defined similarly in Equations 3 and 4.

$$\mathrm{ReLU}(x) \equiv \begin{cases} 0, x < 0 \\ x, x \geq 0 \end{cases} \tag{1}$$

$$\mathrm{tri}(x; l, c, r) = \frac{1}{c-l}\mathrm{ReLU}(x-l) + \frac{1}{r-c}\mathrm{ReLU}(x-r) - \left(\frac{1}{c-l} + \frac{1}{r-c}\right)\mathrm{ReLU}(x-c) \tag{2}$$

$$\mathrm{lshlder}(x; c, r) = \frac{1}{r-c}[\mathrm{ReLU}(r-x) - \mathrm{ReLU}(c-x)] \tag{3}$$

$$\mathrm{rshlder}(x; l, c) = \frac{1}{c-l}[\mathrm{ReLU}(x-l) - \mathrm{ReLU}(x-c)] \tag{4}$$

**Training.** Gradient-based training of the fuzzy trees requires an appropriate parametrization of each FIS. In this work, FIS parametrization consists of 2 parts: (1) the input membership function centers and corners and (2) the rule consequents. For a given input $x$, the centers of the left-shoulder and right-shoulder membership functions are fixed at the left and right extents of $x$'s domain, respectively. With Ruspini partitioning of $N$ membership functions defined over $x$, only $N-2$ remaining membership function centers may vary independently. Once the centers are determined, the corners positions are fixed. Thus, the first part of the parametrization is accomplished with an array of $N-1$ parameters in the interval $[a, b]$ such that $0 < a \ll b$. The ordered array of remaining membership function centers are extracted from these parameters by taking the cumulative sum and normalizing the first $N-2$ values by the $N-1$th value. This produces a sorted array of strictly increasing centers in the interval $(0,1)$. The 0-order rule consequents are readily parametrized with 1 parameter per rule. A separate set of parameters is generated for each FIS. Model 1 contains 360 trainable parameters; model 2 contains 673.

In this work, each model was trained on the breast cancer dataset with the Adam optimizer and cross entropy loss implemented in PyTorch. For each model, a learning rate of $0.01$ with 10 minibatches and less than 20 epochs was found to be sufficient to achieve high accuracy on train and test sets. A 80%-20% train-test split was used.

### 2.4 Bound Generation

The verification approaches used in this work can generate upper and lower bounds for a model's outputs given a bounded input set. Here, a bounded input set $\Sigma_{\varepsilon,y}$ is defined as an ε-perturbation about an input vector $y$ (Eqn. 5). The upper and lower bounds, $\mathrm{lb}(x)$ and $\mathrm{ub}(x)$ (Eqn. 6), respectively, can be either constant or linear functions of the input $x$. Two algorithms are used to generate the output bounds. The first, CROWN, computes the output bounds directly. The second, α-CROWN, extends the functionality of CROWN by iteratively refining the bounds when the bounds are not uniquely determined.

$$\Sigma_{\varepsilon,y} \equiv \{x | \varepsilon \geq \|x-y\|_\infty\} \text{ where } \|z\|_\infty = \max_i |z_i| \tag{5}$$

$$\mathrm{lb}(x) \leq \mathrm{model}(x) \leq \mathrm{ub}(x)\ \forall x \in \Sigma_{\varepsilon,y} \tag{6}$$

When checking robustness of a classifier with multiple outputs, one may check whether a given datapoint is classified correctly when subject to small perturbations of the input features. If so, this grants some confidence that the classifier is more robust than one which does not exhibit this property. In this work, the entire dataset is checked for perturbation robustness. As mentioned above, the final classification of a datapoint is made by evaluating which class's output is larger. With two classes, a classification is considered robust under some input perturbation if the difference between the true class's output and the other class's output is guaranteed to be greater than 0. In practice, this is accomplished by defining a new scalar $Z$ (Eqn. 7); the model is robust for a given datapoint if the corresponding lower bound for $Z$ is greater than or equal to 0.

$$Z \equiv \mathrm{out}_{\mathrm{ground_truth}} - \mathrm{out}_{\mathrm{other}} \tag{7}$$

## 3 Results

Loss histories computed during training are shown in Figure 5 for models 1 and 2. Separate loss histories are shown for train and test sets. Early stopping was used to save the maximum-accuracy FIS achieved during training. At the end of training, both models achieved 97.8% test accuracy. On the entire dataset, models 1 and 2 achieved accuracies of 97.7% and 98.5%, respectively.

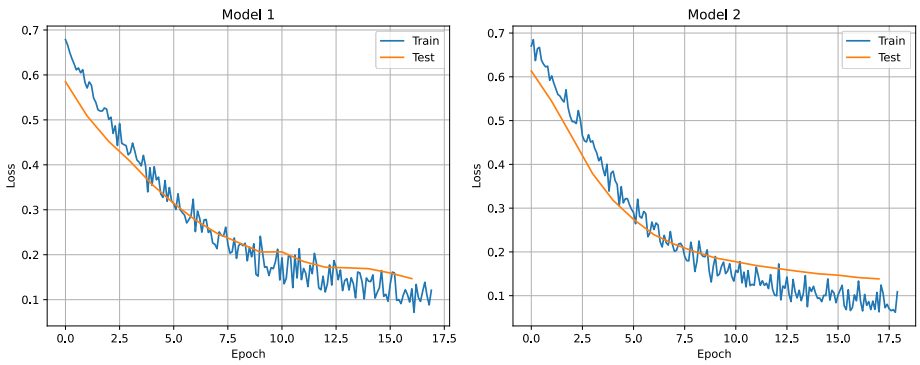


**Fig. 5.** Loss Histories during training for Model 1 (left) and Model 2 (right)

After training, each model was evaluated for robustness over the entire dataset. A model is considered robust at a given datapoint if, when the input is subject to an ε-perturbation about the datapoint, the correct classification can be guaranteed. Each verification approach provides different robustness guarantees depending on the conservativeness of the bounds it generates. Once the entire dataset has been processed, the robustness percentage is calculated as the fraction of datapoints for which the model is robust. Naturally, the robustness percentage changes with the value of ε used for the ε-perturbation. Robustness percentages are shown for a variety of ε values in

Figure 6 for both models. Figure 7 shows the corresponding compute times for both models on a RTX 3080 GPU. Results from both CROWN and α-CROWN are provided.

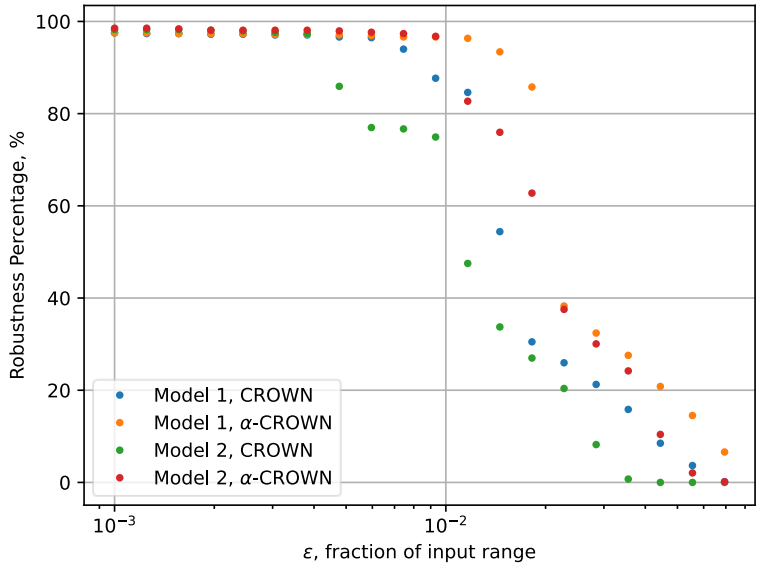


**Fig. 6.** Robustness Percentages of Models 1 and 2 as computed by CROWN and α-CROWN

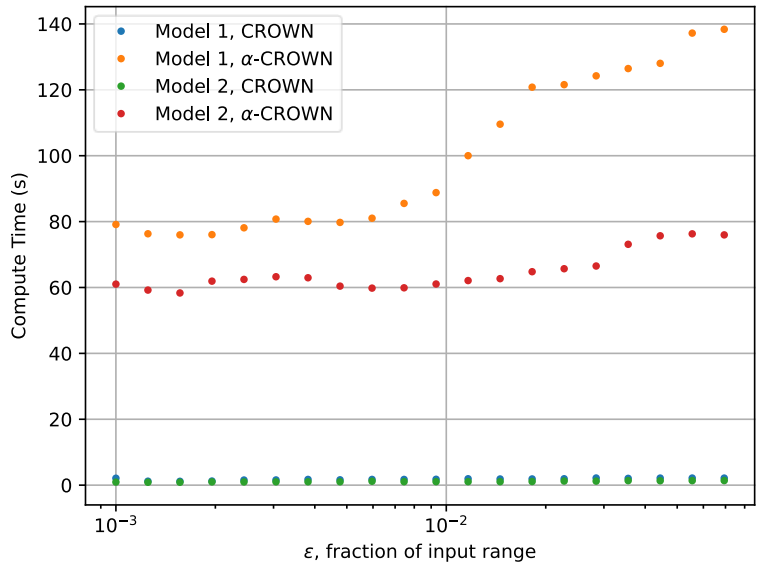


**Fig. 7.** CROWN and α-CROWN Compute Times for Models 1 and 2

## 4 Discussion

Though both models exhibit similar nominal performance, their structures result in significantly different verification results. Whether using CROWN or α-CROWN, model 1 is consistently more robust than model 2, despite model 2's slightly higher baseline accuracy (Fig. 6). This is likely due to fewer FIS inputs on average for model 1 than for model 2, despite model 1 having more FISs overall. Regardless of the verification approach or model, the computed robustness percentage falls below 10% even for relatively small values of $\varepsilon \leq 10\%$ of the input range (Fig. 6). This phenomenon may be due in part to each model's inherent lack of robustness, but it is also due in part to the use of incomplete rather than complete verification. The fact that α-CROWN consistently outperformed CROWN at larger values of ε suggests that a complete verifier would yield further improvements. This may prove the models to be significantly more robust than the current results suggest.

Using CROWN, both models required a similar compute time (Fig. 7). Using α-CROWN, model 1's compute time was significantly larger than model 2's at each value of ε (Fig. 7). This is likely due to the larger number of FISs in model 1, despite a lower overall number of parameters. This suggests a counter-intuitive relationship between computational efficiency at runtime, where model 1 requires fewer operations, and computational efficiency during verification, where model 2 is shown to be faster.

## 5 Conclusions

Two different models consisting of different fuzzy trees are trained on the Breast Cancer Wisconsin (Original) dataset and checked for robustness at each datapoint using state-of-the-art verification techniques CROWN and α-CROWN. The fuzzy trees were represented as ANFIS networks with constrained FISs to facilitate training and the subsequent verification. Both models achieved similar nominal accuracies but substantially different robustness percentages when subjected to perturbed inputs. Consequently, the structure of the model's fuzzy tree is understood to impact both the time required for verification and the resulting robustness estimates. Model 1, with more FISs, more layers, and fewer inputs per FIS than model 2, was verified to be more robust at the expense of additional verification time. Future work will further the understanding of how tree structure impacts verification by testing more fuzzy trees including a single FIS which accepts all 9 inputs.

Regardless of the chosen model, α-CROWN is less conservative than CROWN but requires increased compute time. Following this trend, complete verification techniques are expected to perform better than both CROWN and α-CROWN with additional compute time. Even relatively small input perturbations would benefit from less conservative bounds provided by complete verifiers. Future work will investigate the performance and compute tradeoffs between complete and incomplete verifiers applied to fuzzy trees. In other research, these bound propagation techniques have been applied as an enabling technology for a variety of tasks. For example, computed out-

put bounds have been used for training ML models for improved robustness [17, 18] and reachability analysis of ML-based control [19]. Future work will investigate the application of bound propagation with fuzzy trees to these other tasks.

**Acknowledgements.** We acknowledge the support provided by the Digital Futures Graduate Fellowship, which has provided the requisite resources and funding to enable this research.

## References

1. Arnett, Timothy: Iteratively Increasing Complexity During Optimization for Formally Verifiable Fuzzy Systems. Doctoral dissertation, University of Cincinnati (2019). http://rave.ohiolink.edu/etdc/view?acc_num=ucin156387481300899
2. Arnett, T., Ernest, N., Phillips, Z. (2023). Comparison of Explanation Methods for Genetic Fuzzy Trees for Wine Quality Predictions. In: Cohen, K., Ernest, N., Bede, B., Kreinovich, V. (eds) Fuzzy Information Processing 2023. NAFIPS 2023. Lecture Notes in Networks and Systems, vol 751. Springer, Cham. https://doi.org/10.1007/978-3-031-46778-3_21
3. Fleck DE, Ernest N, Adler CM, Cohen K, Eliassen JC, Norris M, Komoroski RA, Chu WJ, Welge JA, Blom TJ, DelBello MP, Strakowski SM. Prediction of lithium response in first-episode mania using the LITHium Intelligent Agent (LITHIA): Pilot data and proof-of-concept. Bipolar Disord. 2017 Jun;19(4):259-272. doi: 10.1111/bdi.12507. Epub 2017 Jun 2. PMID: 28574156; PMCID: PMC5517343.
4. Pickering, L., Cohen, K. (2022). Toward Explainable AI—Genetic Fuzzy Systems—A Use Case. In: Rayz, J., Raskin, V., Dick, S., Kreinovich, V. (eds) Explainable AI and Other Applications of Fuzzy Techniques. NAFIPS 2021. Lecture Notes in Networks and Systems, vol 258. Springer, Cham. https://doi.org/10.1007/978-3-030-82099-2_31
5. DeGroote, N., Cohen, K. (2022). A Genetic Fuzzy Approach for the Prediction of Heart Failure. In: Rayz, J., Raskin, V., Dick, S., Kreinovich, V. (eds) Explainable AI and Other Applications of Fuzzy Techniques. NAFIPS 2021. Lecture Notes in Networks and Systems, vol 258. Springer, Cham. https://doi.org/10.1007/978-3-030-82099-2_9
6. J. . -S. R. Jang, "ANFIS: adaptive-network-based fuzzy inference system," in IEEE Transactions on Systems, Man, and Cybernetics, vol. 23, no. 3, pp. 665-685, May-June 1993, doi: 10.1109/21.256541.
7. van Oostendorp, B., Zander, E., Bede, B. (2023). Deep Learning ANFIS Architectures. In: Cohen, K., Ernest, N., Bede, B., Kreinovich, V. (eds) Fuzzy Information Processing 2023. NAFIPS 2023. Lecture Notes in Networks and Systems, vol 751. Springer, Cham. https://doi.org/10.1007/978-3-031-46778-3_13
8. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," in IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-15, no. 1, pp. 116-132, Jan.-Feb. 1985, doi: 10.1109/TSMC.1985.6313399.
9. Zhang, H., Weng, T.W., Chen, P.Y., Hsieh, C.J. and Daniel, L., 2018. Efficient neural network robustness certification with general activation functions. *Advances in neural information processing systems*, *31*.
10. Xu, K., Zhang, H., Wang, S., Wang, Y., Jana, S., Lin, X. and Hsieh, C.J., 2020. Fast and complete: Enabling complete neural network verification with rapid and massively parallel incomplete verifiers. *arXiv preprint arXiv:2011.13824*.

11. α,β-CROWN: A Fast and Scalable Neural Network Verifier with Efficient Bound Propagation, https://github.com/Verified-Intelligence/alpha-beta-CROWN, last accessed 2025/04/16.
12. 5th International Verification of Neural Networks Competition (VNN-COMP'24), https://sites.google.com/view/vnn2024, last accessed 2025/04/16.
13. Xu, K., Shi, Z., Zhang, H., Wang, Y., Chang, K.W., Huang, M., Kailkhura, B., Lin, X. and Hsieh, C.J., 2020. Automatic perturbation analysis for scalable certified robustness and beyond. *Advances in Neural Information Processing Systems*, *33*, pp.1129-1141.
14. PyTorch, https://pytorch.org/, last accessed 2025/04/16.
15. O. L. Mangasarian and W. H. Wolberg: "Cancer diagnosis via linear programming", SIAM News, Volume 23, Number 5, September 1990, pp 1 & 18.
16. Enrique H Ruspini. A new approach to clustering. Information and control, 15(1):22{32, 1969.
17. Zhang, H., Chen, H., Xiao, C., Gowal, S., Stanforth, R., Li, B., Boning, D. and Hsieh, C.J., 2019. Towards stable and efficient training of verifiably robust neural networks. *arXiv preprint arXiv:1906.06316*.
18. Shi, Z., Wang, Y., Zhang, H., Yi, J. and Hsieh, C.J., 2021. Fast certified robust training with short warmup. *Advances in Neural Information Processing Systems*, *34*, pp.18335-18349.
19. CROWN-Reach, https://github.com/Verified-Intelligence/CROWN-Reach, last accessed 2025/04/17.